

Compositional Latent Space

Suresh Murugaiyan

July 2026

1 Introduction

This project asks a simple question: when a neural network compresses a flow field into a short list of numbers, can we make those numbers *mean* something physical?

The tool we use is an *autoencoder*: a pair of networks in which an *encoder* squeezes a full flow field (hundreds of thousands of grid values) down to a short vector $\mathbf{z}$, and a *decoder* rebuilds the field from that vector alone. The short vector is called the *latent vector*, and the space it lives in is the *latent space*. Readers who know proper orthogonal decomposition (POD) can think of an autoencoder as a nonlinear POD: instead of modal amplitudes, the network learns its own compressed coordinates. We train on the FlowBench 2D lid-driven cavity dataset: steady Navier–Stokes flow in a square cavity whose top lid moves at constant speed, with a solid object placed inside the cavity.

Ordinarily the latent vector is a black box: the network stores information wherever it likes, and no single entry means anything. Our key structural choice is to split $\mathbf{z}$, by architecture, into three named blocks, each intended to carry exactly one physical factor:

$$\mathbf{z} = [\underbrace{\mathbf{z}_\mu}_{\text{regime (4)}} \parallel \underbrace{\mathbf{z}_g}_{\text{geometry (32)}} \parallel \underbrace{\mathbf{z}_\xi}_{\text{residual (16)}}] \in \mathbb{R}^{52}. \quad (1)$$

In words: 52 numbers summarize each flow field. The first 4 ($\mathbf{z}_\mu$, the *regime* block) should describe the operating condition — here, the Reynolds number Re . The next 32 ($\mathbf{z}_g$, the *geometry* block) should describe the shape of the object in the cavity. The last 16 ($\mathbf{z}_\xi$, the *residual* block) are free capacity for whatever else the reconstruction needs.

The hypothesis under test: after training, $\mathbf{z}_\mu$ should contain the Reynolds number, $\mathbf{z}_g$ should contain the object shape, and neither should *leak* into the other — that is, you should not be able to recover Re by reading the geometry block. Because the flow here is steady (it does not change in time), there is no dynamics block $\mathbf{z}_\eta$ and no latent time-stepper Φ in this first version; those enter later with time-dependent cases.

2 Dataset

This section describes what one training sample looks like and which labels we extract from it. Each FlowBench sample is a steady lid-driven cavity solution stored on a 512×512 grid. The data loader (`data/dataset.py`) unpacks the raw `.npz` arrays into:

- **fields** — the solution itself: horizontal velocity u , vertical velocity v , and pressure p at every grid point. We resize $512^2 \rightarrow 256^2$ to make training cheaper; this loses little information for these smooth steady fields.
- **sdf** — the signed distance field of the object: at every grid point, the distance to the object’s surface, with a positive sign in the fluid and a negative sign inside the solid. The SDF is a convenient, smooth description of the shape, and it serves as our geometry label.
- **mask** — a binary image: 1 where there is fluid, 0 inside the solid object.

- **log_{re}** — the regime label. Re is stored in the file as a constant image, so we read a single pixel to get the number. We use $\log_{10} \text{Re}$ rather than Re because Re spans orders of magnitude, and we *standardize* it (subtract the mean, divide by the standard deviation) so the network sees values of order one. The mean and standard deviation are computed on the *training* set only, and the test set reuses them, so both splits are measured on exactly the same scale.

The loader also computes three simple geometry summaries from the mask: the fraction of the cavity occupied by the solid (area fraction) and the x and y coordinates of the solid’s center (centroid). These three numbers are never used during training. They exist only as ground-truth targets for the diagnostic of Section 5: if the geometry block truly contains the shape, simple shape summaries should be readable from it.

3 Network architecture

Four modules make up the model (`models/compositional/networks.py`):

- **FieldEncoder $\mathcal{E}$** — a convolutional neural network (CNN): a stack of learned filters that repeatedly halves the image resolution ($256 \rightarrow 128 \rightarrow \dots \rightarrow 8$), so that each stage sees increasingly large-scale features of the flow. After the final stage the features are pooled and passed through *three separate linear layers* (“heads”), one producing $\mathbf{z}_\mu$, one $\mathbf{z}_g$, and one $\mathbf{z}_\xi$. The blocks are therefore separate objects from the very start.
- **FieldDecoder $\mathcal{D}$** — the mirror image of the encoder: the full vector $\mathbf{z}$ is reshaped into a small 8×8 feature map and repeatedly upsampled back to a 256^2 (u, v, p) field.
- **RegimeHead** — a small network that is shown $\mathbf{z}_\mu$ *only* and must predict $\log \text{Re}$.
- **SDFHead** — a small decoder that is shown $\mathbf{z}_g$ *only* and must redraw the object’s SDF at 64×64 resolution.

The two small heads are the supervision mechanism, and the logic is worth spelling out. The regime head can only see $\mathbf{z}_\mu$; if it is required to predict Re, then the encoder has no choice but to route Reynolds-number information into that block. Likewise, the SDF head can only see $\mathbf{z}_g$, so shape information must flow there. Note what this does *not* guarantee: it forces the right information *into* the right block, but nothing yet prevents extra copies of the same information from also appearing in the wrong blocks. That gap drives most of the experiments below.

4 Training objective

A *loss* is a number that measures how wrong the network currently is; training is the process of adjusting the network’s parameters to make a weighted sum of losses as small as possible. Each optimization step combines four losses (`models/compositional/compositional_ae.py`), weighted by λ -coefficients set in the configuration file:

1. **Masked reconstruction (L1).** The mean squared error (MSE) between the decoded field and the true field, evaluated over fluid points only. Predictions inside the solid are zeroed out, and the error is divided by the number of fluid points, so a sample with a large object is not unfairly weighted. This loss makes the autoencoder reconstruct accurately.
2. **Regime supervision.** MSE between the regime head’s prediction and the true standardized $\log_{10} \text{Re}$. This pulls Re into $\mathbf{z}_\mu$.
3. **Geometry supervision.** MSE between the SDF head’s output and the true (downsampled) SDF. This pulls the shape into $\mathbf{z}_g$.

4. **Cross-block decorrelation (L6).** The Pearson correlation between two quantities measures their linear relationship: +1 or −1 means perfectly related, 0 means no linear relation. Across each training batch we compute the correlation between every entry of one block and every entry of another block, and penalize its average absolute value, for all three block pairs. The idea: losses 2–3 pull information *into* the correct block; this term is meant to push redundant copies *out of* the wrong blocks by discouraging any linear relationship between blocks.

The residual block z_ξ receives no supervision at all. It is free capacity that absorbs whatever the reconstruction needs beyond (Re, shape) — exactly the role the framework assigns to the residual block.

Training is driven by `main.py`: it loads the YAML configuration, fixes the random seeds (so results are repeatable), builds the train and test datasets (passing the training Re statistics to the test set), and trains with PyTorch Lightning, keeping the checkpoint with the best validation reconstruction error.

5 Linear-probe diagnostics

Training tells us the losses went down; it does not tell us whether the blocks actually mean what we intended. The diagnostic (`diagnostics/probes.py`) is the actual scientific test, and it works as follows. After training, we encode the entire test set. Then, for each block and each known physical quantity, we fit a *linear probe*: a plain linear regression that tries to predict the quantity (say, $\log \text{Re}$) from the entries of one block (say, z_g) alone. We use *ridge* regression (linear regression with a small penalty on the coefficients, which keeps the fit stable) and 5-fold cross-validation (fit on four fifths of the data, test on the remaining fifth, rotate five times, average), so the score reflects genuine predictive power rather than memorization.

The score is R^2 , the fraction of the target’s variance the probe explains: $R^2 = 1$ means the quantity is perfectly readable from the block with a linear formula; $R^2 \approx 0$ means the block carries no usable linear information about it; negative values mean the probe does worse than simply guessing the average. For a compositional latent we expect the block-diagonal pattern of Table 1: each quantity readable from its own block and from nowhere else.

Table 1: Expected linear-probe R^2 pattern for a compositional latent space. High R^2 on the matching block and low R^2 elsewhere indicates a successful factorization; a high off-diagonal entry indicates leakage.

Target	z_μ	z_g	z_ξ
$\log \text{Re}$	high	low	low
Area fraction	low	high	low
Centroid x	low	high	low
Centroid y	low	high	low

If the trained latent reproduces this pattern, the factorization worked: Re is linearly readable from the regime block and *not* from the geometry block, and vice versa. A high R^2 in an off-diagonal cell means information *leaked* into a block where it does not belong — exactly what the loss-ablation experiments below are designed to study.

6 Results

6.1 Overview

Six training runs to date, each changing one ingredient at a time so that any change in the outcome can be attributed to that ingredient. Table 2 summarizes the arc. In brief: the baseline (Run 1) puts the right information in the right blocks but also leaks Re everywhere; making the statistical decorrelation penalty stronger (Runs 2–3) trades quality for leakage without fixing the problem; the two *structural* losses — same-factor invariance (L10, Run 4) and swap consistency (L12, Run 5) — deliver clean separation and clean recombination at matched conditions; and the cross-Re swap loss (Run 6) finally delivers *transfer*: the ability to predict a geometry at a new Reynolds number, which every earlier model failed by two orders

of magnitude. Run 6 is the current best configuration. The following subsections document each run in chronological order, each ending with the reasoning that led to the next run.

Table 2: Summary of all runs. “Leakage” is the linear-probe R^2 of log Re read from $\mathbf{z}_g$, where it does not belong (target $\lesssim 0.3$); the swap ratios compare swap error to reconstruction error (target ≈ 1 ; defined in Section 6.7).

Run	Configuration	Leakage	Same-Re	Cross-Re	Verdict
1	$\lambda_{\text{decorr}} = 0.01$	0.900	1.06	260	Re leaks into all blocks
2	$\lambda_{\text{decorr}} = 0.5$	0.719	—	—	trade-off: quality lost, leak remains
3	$\lambda_{\text{decorr}} = 0.1$	0.865	—	—	statistical penalty insufficient
4	+ invariance L10	0.215	1.35	194	separated, but brittle
5	+ swap L12	0.343	1.04	199	recombinable, no transfer
6	+ cross-Re swap	0.405	1.57	2.1	<i>transfer learned</i>

6.2 Run 1: baseline loss weights

First training run on the FlowBench LDC “easy” split (2,400 training / 600 test samples): 200 epochs (an epoch is one full pass through the training data), batch size 16, resolution 256^2 , loss weights $\lambda_{\text{recon}} = 1.0$, $\lambda_{\text{regime}} = \lambda_{\text{geo}} = 0.1$, $\lambda_{\text{decorr}} = 0.01$ (single NVIDIA GPU, ~ 18 minutes). The final masked reconstruction error was 2.7×10^{-3} on the training set and 2.1×10^{-3} on the validation set — similar values, so the network is not merely memorizing the training data. Table 3 reports the measured linear-probe R^2 .

Table 3: Run 1: measured linear-probe R^2 on the 600-sample test split ($\lambda_{\text{decorr}} = 0.01$). Bold marks the block that should carry each target.

Target	$\mathbf{z}_\mu$	$\mathbf{z}_g$	$\mathbf{z}_\xi$
log Re	0.994	0.900	0.777
Area fraction	0.042	0.688	0.172
Centroid x	0.027	0.736	0.373
Centroid y	−0.013	0.690	0.214

What worked. The regime block is clean: $\mathbf{z}_\mu$ reads log Re at $R^2 = 0.994$ and contains essentially no geometry ($R^2 \leq 0.04$). Geometry is readable from $\mathbf{z}_g$ ($R^2 \approx 0.69$ – 0.74) and nearly absent from $\mathbf{z}_\mu$. The moderate (rather than near-perfect) geometry scores are not alarming: the probes are *linear*, while area and centroid depend on the encoded shape in a nonlinear way, so even a perfect geometry block would not give $R^2 = 1$ here.

What failed. Reynolds-number information leaked into the other blocks: log Re is readable from $\mathbf{z}_g$ at $R^2 = 0.900$ and from $\mathbf{z}_\xi$ at 0.777. The reason is easy to see. The decoder needs Re to rebuild the field, and nothing stops the encoder from stashing a copy of Re in *every* block — in fact the reconstruction loss rewards extra copies, because they can only help the decoder. A well-known theoretical result (Locatello et al.) says precisely this: without extra constraints, a network that reconstructs well is free to scramble the underlying factors across its latent however it likes. The decorrelation penalty is supposed to prevent the copies, but at $\lambda_{\text{decorr}} = 0.01$ it is roughly a hundred times weaker than the reconstruction term, and it loses the competition.

Next experiment. The leaked Re is *linearly* readable from $\mathbf{z}_g$ — and a linear relationship is exactly what a Pearson penalty acts on. So the next run raises λ_{decorr} from 0.01 to 0.5, changing nothing else. Success would mean: the off-diagonal log Re entries drop substantially (toward $R^2 \lesssim 0.3$) while reconstruction and the geometry row stay near their Run-1 values. If even a strong Pearson weight cannot remove the

leakage, the natural escalation is HSIC (L7, a penalty that detects nonlinear dependence too) and the swap-consistency loss (L12), which attacks the problem functionally rather than statistically.

6.3 Run 2: strong Pearson decorrelation

Identical to Run 1 except $\lambda_{\text{decorr}} = 0.5$, a $50\times$ increase. Table 4 reports the measured linear-probe R^2 , with the Run-1 values in parentheses for comparison.

Table 4: Run 2: measured linear-probe R^2 on the test split ($\lambda_{\text{decorr}} = 0.5$), with Run-1 values ($\lambda_{\text{decorr}} = 0.01$) in parentheses. Bold marks the block that should carry each target.

Target	$\mathbf{z}_\mu$	$\mathbf{z}_g$	$\mathbf{z}_\xi$
log Re	0.886 (0.994)	0.719 (0.900)	0.487 (0.777)
Area fraction	-0.002 (0.042)	0.402 (0.688)	0.262 (0.172)
Centroid x	-0.016 (0.027)	0.565 (0.736)	0.378 (0.373)
Centroid y	0.016 (-0.013)	0.427 (0.690)	0.115 (0.214)

Outcome: a trade-off, not a fix. The leakage did shrink — log Re from $\mathbf{z}_g$ dropped from 0.900 to 0.719, and from $\mathbf{z}_\xi$ from 0.777 to 0.487 — but everything else got worse too: the on-diagonal regime readout degraded (0.994 $\rightarrow$ 0.886) and geometry recovery from $\mathbf{z}_g$ fell by roughly 0.2 on all three descriptors. This is what economists call a *Pareto trade-off*: one goal improves only by sacrificing another, with no setting that improves both. The underlying reason is that the Pearson penalty is a blunt instrument. It cannot tell the difference between a redundant *copy* of information (which we want removed) and informative, structured coordinates (which we want kept); at a large weight it simply flattens all correlations, scrambling every block a little — while the leakage still sits far above the $R^2 \lesssim 0.3$ target. A $50\times$ stronger penalty bought a modest leakage reduction at a real cost: strong evidence that tuning the weight of a statistical independence penalty will not, by itself, separate the blocks.

Next experiments. Two directions, in order: (i) $\lambda_{\text{decorr}} = 0.1$ as an intermediate point — with Runs 1–2 it maps out how the trade-off curve behaves between the two extremes; (ii) *same-factor invariance* (L10). The dataset contains each geometry simulated at several Reynolds numbers, and that structure can be exploited directly: penalize any *variation* of $\mathbf{z}_g$ across samples that share a geometry. Since those samples differ only in Re, this demands outright that $\mathbf{z}_g$ not change when only Re changes — a *functional* constraint on the encoder’s behavior, rather than a statistical property of a batch.

6.4 Run 3: intermediate decorrelation — the verdict on the sweep

Identical to Runs 1–2 except $\lambda_{\text{decorr}} = 0.1$. Table 5 assembles the complete three-point weight sweep in one place.

Verdict on statistical decorrelation. The intermediate weight keeps nearly all of Run 1’s quality (regime readout 0.978, geometry 0.66–0.71) but buys almost no leakage reduction (log Re from $\mathbf{z}_g$: 0.900 $\rightarrow$ 0.865). Across the whole sweep, leakage falls only when representation quality is heavily taxed: there is *no* weight at which the Pearson penalty removes the redundant Reynolds copy while preserving what the blocks are supposed to contain. This is a clean negative result, and it matches the framework’s prediction: statistical independence penalties cannot make a factorization *functional*. The next run therefore switches to the structural tool.

Setting up Run 4: same-factor invariance (L10). Because each geometry appears at several Reynolds numbers, we can build *group-structured minibatches*: instead of sampling training batches at random, each batch is deliberately composed of a few geometry groups, each contributing the same shape at several different Re. The new loss is

$$\mathcal{L}_{\text{inv}} = \mathbb{E}_{\text{groups}} \left[\left\| \mathbf{z}_g^{(i)} - \overline{\mathbf{z}_g}^{\text{group}} \right\|^2 \right], \quad (2)$$

Table 5: Pearson-decorrelation weight sweep: linear-probe R^2 on the test split for $\lambda_{\text{decorr}} \in \{0.01, 0.1, 0.5\}$ (Runs 1, 3, 2). Bold marks the block that should carry each target.

Target	λ_{decorr}	z_μ	z_g	z_ξ
log Re	0.01	0.994	0.900	0.777
	0.1	0.978	0.865	0.752
	0.5	0.886	0.719	0.487
Area fraction	0.01	0.042	0.688	0.172
	0.1	0.017	0.657	0.368
	0.5	-0.002	0.402	0.262
Centroid x	0.01	0.027	0.736	0.373
	0.1	0.014	0.712	0.493
	0.5	-0.016	0.565	0.378
Centroid y	0.01	-0.013	0.690	0.214
	0.1	0.012	0.657	0.245
	0.5	0.016	0.427	0.115

the average squared distance between each sample’s geometry code and the mean geometry code of its group. Samples within a group differ *only* in Re; so if z_g contains any Re content at all, it must vary within the group, and this loss punishes it directly. Run 4 keeps $\lambda_{\text{decorr}} = 0.01$ (the Run-1 value) and adds only the invariance term, so any improvement is attributable to L10 alone.

6.5 Run 4: same-factor invariance — the structural loss succeeds

Run 4 uses the Run-1 weights plus the same-factor invariance term with $\lambda_{\text{inv}} = 0.1$, trained on group-structured minibatches (four geometry groups of four Reynolds numbers per batch of 16). Table 6 reports the probe R^2 against the Run-1 baseline.

Table 6: Run 4: linear-probe R^2 with the same-factor invariance loss (L10), Run-1 values in parentheses. Bold marks the block that should carry each target.

Target	z_μ	z_g	z_ξ
log Re	0.986 (0.994)	0.215 (0.900)	0.412 (0.777)
Area fraction	-0.003 (0.042)	0.677 (0.688)	0.130 (0.172)
Centroid x	0.009 (0.027)	0.756 (0.736)	0.143 (0.373)
Centroid y	0.011 (-0.013)	0.711 (0.690)	0.047 (0.214)

Outcome. The Reynolds leakage into the geometry block collapsed from $R^2 = 0.900$ to 0.215 — below the 0.3 target — at essentially no cost: the regime readout is intact (0.986) and the geometry recovery is even marginally *better* than the baseline (0.68–0.76). The residual block also cleaned up substantially (0.777 $\rightarrow$ 0.412) although nothing constrains it directly; the likely explanation is that once z_g is forced to be Re-free, the decoder comes to rely on z_μ for the operating point, and the now-useless copies elsewhere fade. Contrast this with the weight sweep of Table 5, where a $50\times$ stronger statistical penalty could not achieve the same thing at any acceptable price. This is the framework’s central claim demonstrated on real data: *functional* constraints — invariance within factor groups, made possible by the fact that the dataset was generated on a designed (Re, shape) grid — separate the blocks where *statistical* independence penalties cannot. The probe matrix now shows the intended block-diagonal pattern for both supervised blocks.

Remaining gap and next experiments. The residual block still carries moderate Re content ($R^2 = 0.412$). Natural next steps: (i) *swap consistency* (L12) — decode the regime code of one sample together

with the geometry code of another and compare against the true field for that combination, upgrading the demand from “blocks are separate” to “blocks are *recombinable*”; (ii) concept-vector arithmetic on $\mathbf{z}_\mu$ and $\mathbf{z}_g$; (iii) shrinking the residual block, since a steady, deterministic dataset arguably needs little residual capacity.

6.6 Run 5: adding swap consistency

Run 5 adds the swap-consistency loss (L12) on top of the Run-4 configuration ($\lambda_{\text{swap}} = 0.1$). The idea: if two samples i and k sit at the *same* Reynolds number but have *different* geometries — and 87% of training samples have such a partner — then the decoder should be able to rebuild sample k ’s field from the hybrid latent $[\mathbf{z}_\mu^{(i)} \parallel \mathbf{z}_g^{(k)} \parallel \mathbf{z}_\xi^{(k)}]$, i.e. with k ’s own geometry and residual codes but i ’s regime code. Since i and k share the same Re, a truly compositional model should not care whose regime code it gets: the regime code must be *functionally interchangeable* across geometries at the same operating condition. Table 7 reports the probes, with Run-4 values in parentheses.

Table 7: Run 5: linear-probe R^2 with invariance (L10) plus swap consistency (L12); Run-4 values (L10 only) in parentheses. Bold marks the block that should carry each target.

Target	$\mathbf{z}_\mu$	$\mathbf{z}_g$	$\mathbf{z}_\xi$
log Re	0.972 (0.986)	0.343 (0.215)	0.356 (0.412)
Area fraction	−0.005 (−0.003)	0.693 (0.677)	0.170 (0.130)
Centroid x	−0.003 (0.009)	0.767 (0.756)	0.235 (0.143)
Centroid y	0.003 (0.011)	0.718 (0.711)	0.069 (0.047)

Outcome. The block-diagonal pattern is preserved; the predicted cleanup of the residual block occurred (Re from $\mathbf{z}_\xi$: 0.412 $\rightarrow$ 0.356 — swap decoding pairs the target’s residual with a foreign regime code, so Re hidden in the residual gets punished); and geometry recovery is the best of all runs (0.69–0.77). One cell moved the wrong way: Re from $\mathbf{z}_g$ rose from 0.215 to 0.343. Differences of this size between two runs that differ in random initialization should not be over-interpreted; deciding whether the change is real requires repeating each configuration with several random seeds and comparing averages (the multi-seed protocol of the project plan). The honest summary at this stage: *same-factor invariance (L10) is the decisive ingredient for block separation; the effect of L12 on the probe matrix is small and within plausible run-to-run variation.*

What the probes cannot see. A linear probe measures whether a block *contains* the right information. It says nothing about whether the decoder can actually *recombine* blocks from different samples — which is precisely the property L12 trains. To measure that, we need a different instrument: encode two samples, decode the hybrid latent, and compare against the true field for that combination. That swap-error metric is built next.

6.7 Swap-error diagnostic: separation is not recombination

The swap-error metric works on the trained model without any retraining: for same-Re, different-geometry test pairs, decode the hybrid latent $[\mathbf{z}_\mu^{(i)} \parallel \mathbf{z}_g^{(k)} \parallel \mathbf{z}_\xi^{(k)}]$ and measure its error against sample k ’s true field. We report it as a *ratio* to the ordinary reconstruction error on the same samples: ratio ≈ 1 means swapping in another sample’s regime code costs nothing extra; a large ratio means the decoder secretly cares which sample the regime code came from. Because no retraining is needed, the metric was evaluated retroactively on the three key stored checkpoints (Table 8).

Interpretation. The three numbers tell a story the probe tables could not. Run 1 passes the swap test (1.06) *for the wrong reason*: its decoder held redundant Re copies in every block, so it barely depended on which sample’s $\mathbf{z}_\mu$ it received — if you ignore an input, swapping that input is painless. Run 4’s training removed the redundancy, which made the decoder genuinely dependent on $\mathbf{z}_\mu$; now the small sample-to-sample differences in the regime code propagate to the output (1.35). In other words, *separation alone makes*

Table 8: Same-Re swap-consistency error on the test split (256 pairs), evaluated retroactively on stored checkpoints.

Run	Losses	Probe separation	Swap ratio
Run 1	reconstruction + weak L6	poor (Re everywhere)	1.06
Run 4	+ invariance (L10)	good	1.35
Run 5	+ swap (L12)	good	1.04

the factorization brittle. Run 5’s swap loss trains exactly that sensitivity away, restoring the ratio to 1.04 while keeping the probe separation. The combination — blocks that are both separated *and* functionally interchangeable — requires both structural losses, and neither the probe matrix nor the swap ratio alone certifies both properties. Run 5 is the best configuration overall.

6.8 Cross-Re swap: the transfer test fails — for every model

The same-Re swap is a consistency check, but a weak one: regime supervision makes $\mathbf{z}_\mu$ almost a pure function of Re, so two same-Re samples have nearly identical regime codes, and swapping them is a small perturbation. The demanding test is the *cross-Re* swap: take $\mathbf{z}_\mu$ from a sample at Re_i , take $\mathbf{z}_g, \mathbf{z}_\xi$ from a *different* geometry observed at a *different* Re, decode, and compare against the true field of that geometry at Re_i — which exists in the dataset, so we have exact ground truth. This is genuine *transfer*: using the regime code to move a geometry to an operating point where the model never saw that particular latent combination. Table 9 reports the measurement on the three stored checkpoints.

Table 9: Same-Re vs. cross-Re swap ratios (swap error relative to reconstruction error, 256 pairs, test split).

Run	Losses	Same-Re ratio	Cross-Re ratio
Run 1	baseline	1.06	260
Run 4	+ L10	1.35	194
Run 5	+ L10 + L12	1.04	199

Outcome: falsified. Every model fails the transfer test catastrophically: cross-Re swap errors are two hundred times the reconstruction error, barely improved by the structural losses. So the blocks are separated (probes pass), and interchangeable at matched conditions (same-Re swap passes), yet the decoder cannot *compose* a regime code with a geometry it never saw at that operating point. Strong compositionality — the transfer ability the framework ultimately wants — does not emerge from separation and same-Re consistency alone. This is the diagnostic doing its job: we made a falsifiable prediction, and it was falsified. It also sharpens the question of *mechanism*: the residual block still carries Re ($R^2 \approx 0.36$), and in the cross-Re swap the residual comes from the donor geometry at the *wrong* Re — so the decoder may be reading the operating point from the residual rather than from $\mathbf{z}_\mu$. To test exactly this, the diagnostic now also reports the swapped output’s error against the donor’s own field: if that error is small, the decoder followed the residual’s Re and ignored the regime code.

Run 6: cross-Re swap as a training loss. The dataset contains the required cross-combinations, so the transfer demand can be trained directly rather than merely tested. For in-batch triples (i, k, m) with $\text{re}_m = \text{re}_i$, $\text{geo}_m = \text{geo}_k \neq \text{geo}_i$ and $\text{re}_k \neq \text{re}_i$, decoding $[\mathbf{z}_\mu^{(i)} \parallel \mathbf{z}_g^{(k)} \parallel \mathbf{z}_\xi^{(k)}]$ must reproduce sample m — geometry k moved to operating point Re_i . This forces the decoder to take the operating point from $\mathbf{z}_\mu$ and to treat $(\mathbf{z}_g, \mathbf{z}_\xi)$ as Re-free, attacking the residual leakage as a side effect. Run 6 adds this term ($\lambda_{\text{xswap}} = 0.1$) to the Run-5 configuration.

6.9 Tracking the residual block: improved, but not fixed

One thread runs through all the results above and deserves its own summary: the Reynolds content of the residual block. Recall that $\mathbf{z}_\xi$ is deliberately unsupervised — it is free storage for whatever the reconstruction needs beyond (Re, shape) — so nothing has ever told it *not* to hold a copy of Re. Table 10 shows how much Re is readable from it after each run (target: near zero).

Table 10: Reynolds content of the residual block across runs: linear-probe R^2 of log Re read from $\mathbf{z}_\xi$. The ideal value is near 0.

Run	Losses	log Re from $\mathbf{z}_\xi$
Run 1	baseline	0.777
Run 4	+ L10	0.412
Run 5	+ L10 + L12	0.356
Run 6	+ cross-Re swap	0.243

Through Run 5 the leakage had dropped by more than half, yet $R^2 = 0.356$ still meant that roughly a third of the Reynolds information sat in a block where it does not belong. It is worth noticing that neither L10 nor L12 targets $\mathbf{z}_\xi$ directly; those improvements were *side effects*. Once L10 forced $\mathbf{z}_g$ to be Re-free, the decoder had to rely more on $\mathbf{z}_\mu$ for the operating point, and the now-less-useful copies elsewhere partly decayed.

Why this matters beyond tidiness: the leftover Re in $\mathbf{z}_\xi$ was our prime suspect for the cross-Re transfer failure of Table 9. In the cross-Re swap, the residual code comes from the donor sample at the *wrong* Reynolds number; if the decoder reads the operating point from the residual instead of from $\mathbf{z}_\mu$, transfer breaks in exactly the way we observed.

Run 6 was the first run to attack this directly. Its cross-Re training loss decodes $[\mathbf{z}_\mu^{(i)} \parallel \mathbf{z}_g^{(k)} \parallel \mathbf{z}_\xi^{(k)}]$ against ground truth at Re_i , so any Re content in $\mathbf{z}_\xi^{(k)}$ (which is at the wrong Re) actively increases the loss, and training pressure squeezes it out. As the last row of Table 10 shows, it worked: the residual’s Re content fell to 0.243, the lowest of any run — and, as the next subsection reports, the transfer failure fell with it. If further cleanup is needed, two follow-ups remain queued: shrink the residual block from 16 to 4 dimensions (less room to hide Re), and the HSIC penalty (which catches nonlinear dependence that the Pearson penalty misses).

6.10 Run 6: the cross-Re swap loss — transfer learned

Run 6 adds the cross-Re swap term ($\lambda_{\text{xswap}} = 0.1$) to the Run-5 configuration and trains from scratch, with everything else unchanged. Table 11 reports the probes (Run-5 values in parentheses), and the swap diagnostics follow.

Table 11: Run 6: linear-probe R^2 with the cross-Re swap loss added; Run-5 values in parentheses. Bold marks the block that should carry each target.

Target	$\mathbf{z}_\mu$	$\mathbf{z}_g$	$\mathbf{z}_\xi$
log Re	0.987 (0.972)	0.405 (0.343)	0.243 (0.356)
Area fraction	0.148 (−0.005)	0.673 (0.693)	0.328 (0.170)
Centroid x	−0.011 (−0.003)	0.749 (0.767)	0.165 (0.235)
Centroid y	0.013 (0.003)	0.696 (0.718)	0.038 (0.069)

The headline: transfer works now. The cross-Re swap ratio collapsed from 199 to **2.12** — from two hundred times the reconstruction error to about twice it. In absolute terms the cross-combination fields are decoded with error 4.0×10^{-3} , close to ordinary reconstruction quality. The model went from “cannot transfer at all” to “predicts a geometry at a new Reynolds number almost as well as it reconstructs a field it has seen.”

The mechanism test confirms the diagnosis. Recall the suspicion that earlier decoders were reading the operating point from the residual code rather than from z_μ . The new donor comparison settles it: the swapped output’s error against the *target* (geometry k at the new Re_i) is 4.0×10^{-3} , while its error against the *donor* (geometry k at its own, wrong Re) is 0.584 — more than a hundred times larger. The decoded field follows the regime code, not the residual: the decoder now takes its operating point from z_μ , exactly as designed. Consistently, the residual’s own Re content dropped to its best value ($0.356 \rightarrow 0.243$; Table 10), and the regime readout even improved ($0.972 \rightarrow 0.987$).

The honest fine print. Three cells moved slightly the wrong way: the same- Re swap ratio drifted from 1.04 to 1.57, Re from z_g rose from 0.343 to 0.405, and small amounts of geometry appeared in z_μ (area fraction 0.148) and z_ξ (0.328). All of these are small compared to the two-orders-of-magnitude transfer improvement, and all are of the size that single-seed comparisons cannot resolve; the multi-seed protocol will determine which are real effects and which are run-to-run noise. The overall verdict stands: *with the cross- Re swap loss, and a dataset whose (Re , shape) grid supplies ground truth for cross-combinations, transfer compositionality is directly trainable — at essentially no cost to reconstruction or to the block structure.*

6.11 Multi-seed results: what is robust and what was noise

Every result above came from a single training run per configuration. Neural-network training is stochastic — the random initialization of the weights changes the outcome — so small differences between two runs may be luck rather than effect. To separate the two, the Run-4, Run-5, and Run-6 configurations were each retrained with three different random seeds, and every metric is reported as mean $\pm$ standard deviation across the three seeds (Table 12).

Table 12: Multi-seed results: mean $\pm$ standard deviation over three random seeds per configuration. Bold marks the best value in each row. [†]Average of the three geometry-descriptor probes (area fraction, centroid x , centroid y).

Metric	Run 4 (L10)	Run 5 (+L12)	Run 6 (+cross- Re)
log Re from z_μ (want high)	0.946 ± 0.029	0.930 ± 0.042	0.984 ± 0.002
log Re from z_g (want low)	0.284 ± 0.052	0.315 ± 0.084	0.376 ± 0.074
log Re from z_ξ (want low)	0.247 ± 0.128	0.307 ± 0.130	0.200 ± 0.041
Geometry from z_g (want high) [†]	0.733 ± 0.013	0.701 ± 0.042	0.732 ± 0.018
Same- Re swap ratio (want ≈ 1)	1.36 ± 0.23	1.04 ± 0.03	1.50 ± 0.05
Cross- Re swap ratio (want ≈ 1)	171 ± 66	163 ± 73	2.0 ± 0.1

What survives with error bars. Four conclusions are now statistically solid. *First*, the transfer result is rock-solid: Run 6 achieves a cross- Re ratio of 2.0 ± 0.1 in every seed, while Runs 4 and 5 fail by factors of 60–240 in every seed — the distributions do not remotely overlap. *Second*, L12’s effect on same- Re recombability is real, not noise: Run 5 sits at 1.04 ± 0.03 (remarkably tight) versus Run 4’s noisy 1.36 ± 0.23 . *Third*, Run 6’s same- Re cost is also real (1.50 ± 0.05 , consistent across seeds): the cross- Re loss trades a little same- Re polish for transfer. *Fourth*, Run 6 is the most *stable* configuration — its regime readout (0.984 ± 0.002) varies ten times less across seeds than Runs 4–5, and its residual block is the cleanest.

What was noise. The z_g leakage differences that earlier subsections cautiously flagged (0.215 vs. 0.343 vs. 0.405) are indeed within run-to-run variation: all three configurations overlap around 0.28–0.38, and Run 4’s celebrated seed-0 value of 0.215 was partly luck. The honest multi-seed claim is that the structural losses hold the leakage near ~ 0.3 (against the 0.900 baseline), with no significant difference among Runs 4–6.

The mechanism evidence also generalizes. Across the Run-4/5 seeds, the cross- Re swapped output often lay *closer to the donor’s field than to the target’s* (donor error 0.07–0.11 versus target error 0.34–

0.40) — direct proof that those decoders read the operating point from $\mathbf{z}_g$ or $\mathbf{z}_\xi$. In every Run-6 seed the relationship is reversed (donor ≈ 0.58 , target ≈ 0.003): the decoder follows $\mathbf{z}_\mu$, exactly as designed.

7 Status and next steps

Implemented and validated. The block-structured autoencoder with regime and geometry supervision (Runs 1–3); group-structured minibatches and the same-factor invariance loss L10 (Run 4); the swap-consistency loss L12 (Run 5); and a diagnostic suite comprising linear probes, same-Re swap error, and cross-Re swap error — all runnable on stored checkpoints without retraining. Also complete: the cross-Re swap promoted from diagnostic to training loss (Run 6), which took the transfer test from a two-orders-of-magnitude failure to near reconstruction quality. The multi-seed protocol (three seeds per configuration, Table 12) is also complete and confirms the headline findings with error bars: *statistical decorrelation cannot functionally separate the blocks at any weight; the structural losses achieve separation (L10), recombability (L12, 1.04 ± 0.03), and transfer (cross-Re swap, 2.0 ± 0.1 versus ~ 170 without it) at essentially no cost to reconstruction*. The three properties are genuinely distinct: each required its own loss, and each is invisible to the other properties’ diagnostics.

Planned increments, in order.

- **Concept-vector arithmetic** on $\mathbf{z}_\mu$ and $\mathbf{z}_g$: walk the latent along the direction associated with one parameter and check that only that parameter responds (the AeroJEPA-style sensitivity matrix; a Week-15 success criterion of the project plan).
- **Residual capacity**: shrink the residual block (e.g. $16 \rightarrow 4$ dimensions) — a steady, deterministic dataset needs little residual capacity, and less room may squeeze out the Re content remaining in $\mathbf{z}_\xi$ ($R^2 \approx 0.36$).
- **Loss library extensions**: HSIC (L7), which detects nonlinear dependence between blocks, not just linear correlation; boundary-layer weighted reconstruction (L3), which weights errors near the object surface more heavily.
- **Architecture**: an INR-style implicit decoder, which can be queried at any spatial point rather than on a fixed grid.
- **Beyond steady flow**: the dynamics block $\mathbf{z}_\eta$ and a latent time-stepper Φ for time-dependent cases (the framework’s Path B/C), moving toward the warm-up and Case-1 studies of the anchor paper.

A Appendix: Running the code on Nova cluster

1. Log in to the Nova cluster: `ssh username@nova.its.iastate.edu`
2. Activate the virtual environment that contains the required packages.
3. Request a resource allocation: `salloc -N 1 -n 4 -t 01:00:00 --gres=gpu:a100 --mem 369G`
4. Train: `python main.py --config configs/compositional/conf.yaml`
5. Run the diagnostics on a checkpoint (linear probes and swap errors; `version_N` is printed at the end of training):

```
python diagnostics/probes.py \  
  --config configs/compositional/conf.yaml \  
  --checkpoint checkpoints/compositional/version_N/last.ckpt
```